

Coding & Solution

Project: EduGenie: Google Gemini Powered Learning Assistant

Team ID	Team Size	Team Leader	Team Members
SWTID-2026-5228	5	Madhubala G	Pavithra G, Logeswari G, Sivapriya W, Gopika M

Modules and Solution Approach

Module	Function	Description
Doubt Solver	explain_topic(question, level)	Builds a tutor prompt with the chosen level and sends it to Gemini
Summarizer	summarize_notes(text)	Asks Gemini for short revision bullet points
Quiz Generator	generate_quiz(topic, n)	Asks for n MCQs with options, answers and explanations
Study Planner	create_study_plan(subject, days, hours)	Asks for a day-wise plan with revision and practice
Core call	ask_gemini(prompt)	One reusable function for all Gemini requests, with error handling
UI	app.py	Tabs, input validation, spinner and session chat history

gemini_utils.py

```
"""Gemini helper functions for EduGenie."""
import os

from dotenv import load_dotenv
from google import genai

load_dotenv()

# Model name can be changed in the .env file without touching the code.
MODEL = os.getenv("GEMINI_MODEL", "gemini-3.5-flash")

_client = None

def get_client():
    """Create the Gemini client once and reuse it."""
    global _client
    if _client is None:
        api_key = os.getenv("GEMINI_API_KEY")
        if not api_key:
            raise ValueError("GEMINI_API_KEY is missing. Add it to your .env file.")
        _client = genai.Client(api_key=api_key)
    return _client

def ask_gemini(prompt):
    """Send a prompt to Gemini and return the text, or a friendly error."""
    try:
        response = get_client().models.generate_content(model=MODEL, contents=prompt)
        return response.text
    except ValueError as err:
        return "Error: " + str(err)
    except Exception as err:
        return "Sorry, Gemini could not answer right now. Details: " + str(err)

def explain_topic(question, level):
    prompt = (
        "You are a friendly tutor. Explain the following to a "
        + level
        + " level student in simple language, with one example.\n\n"
```

```

        + "Question: "
        + question
    )
    return ask_gemini(prompt)

def summarize_notes(text):
    prompt = (
        "Summarize the following study notes into short, clear bullet points "
        "that are easy to revise:\n\n" + text
    )
    return ask_gemini(prompt)

def generate_quiz(topic, num_questions=5):
    prompt = (
        "Create " + str(num_questions) + " multiple choice questions on '" + topic + "'. "
        "Give 4 options (A to D) for each question. After each question, write the "
        "correct answer and a one-line explanation."
    )
    return ask_gemini(prompt)

def create_study_plan(subject, days, hours_per_day):
    prompt = (
        "Create a day-wise study plan for " + subject + " for " + str(days) + " days, "
        "with " + str(hours_per_day) + " hours of study per day. "
        "Include topics for each day, a short revision slot and one practice task."
    )
    return ask_gemini(prompt)

```

app.py

```

"""EduGenie - Google Gemini Powered Learning Assistant (Streamlit UI)."""
import streamlit as st

from gemini_utils import (
    create_study_plan,
    explain_topic,
    generate_quiz,
    summarize_notes,
)

st.set_page_config(page_title="EduGenie")
st.title("EduGenie")
st.caption("Google Gemini Powered Learning Assistant")

if "history" not in st.session_state:
    st.session_state.history = []

tab_ask, tab_sum, tab_quiz, tab_plan = st.tabs(
    ["Ask a Doubt", "Summarizer", "Quiz", "Study Plan"]
)

with tab_ask:
    level = st.selectbox("Your level", ["beginner", "intermediate", "advanced"])
    question = st.text_area("Type your question")
    if st.button("Explain", key="btn_explain"):
        if not question.strip():
            st.warning("Please enter a question.")
        else:
            with st.spinner("Thinking..."):
                answer = explain_topic(question, level)
                st.session_state.history.append((question, answer))
                st.write(answer)

with tab_sum:
    notes = st.text_area("Paste your notes here", height=200)
    if st.button("Summarize", key="btn_summary"):
        if not notes.strip():
            st.warning("Please paste some notes.")
        else:
            with st.spinner("Summarizing..."):
                st.write(summarize_notes(notes))

with tab_quiz:
    topic = st.text_input("Quiz topic")
    count = st.slider("Number of questions", 3, 10, 5)

```

```

if st.button("Generate Quiz", key="btn_quiz"):
    if not topic.strip():
        st.warning("Please enter a topic.")
    else:
        with st.spinner("Creating quiz..."):
            st.write(generate_quiz(topic, count))

with tab_plan:
    subject = st.text_input("Subject")
    days = st.number_input("Number of days", min_value=1, max_value=60, value=7)
    hours = st.number_input("Hours per day", min_value=1, max_value=12, value=2)
    if st.button("Create Plan", key="btn_plan"):
        if not subject.strip():
            st.warning("Please enter a subject.")
        else:
            with st.spinner("Planning..."):
                st.write(create_study_plan(subject, int(days), int(hours)))

with st.sidebar:
    st.header("Chat History")
    if not st.session_state.history:
        st.write("No questions yet.")
    for q, a in reversed(st.session_state.history):
        with st.expander(q[:40]):
            st.write(a)

```